AdotaPet

Lares que esperam, histórias que começam aqui.

MVP web full stack para ONGs e abrigos gerenciarem os animais disponíveis para adoção.

Angular 22 | Spring Boot 4 | JWT | PostgreSQL

EQUIPE

TB — **Társis Carvalho Barreto** — Matrícula 202220038

YB — **Ylo Silva de Sá Bittencourt** — Matrícula 202220039

GF — **Guilherme Crocamo Freire** — Matrícula 202220448

SS — **Samuel Ferreira Santos** — Matrícula 202410518

JO — **João Pedro França Chaves Oliveira** — Matrícula 202320100

Desenvolvimento de um MVP web full stack — Parte 1

O PROBLEMA

Adoção responsável não cabe em uma planilha.

ONGs e protetores independentes controlam os animais em **planilhas, cadernos e grupos de mensagens**. O resultado é previsível:

Informação espalhada

Dados de saúde, idade e porte ficam em lugares diferentes e se perdem com a troca de voluntários.

Divulgação desatualizada

Animais já adotados continuam sendo anunciados, gerando retrabalho e frustração de quem quer adotar.

Sem histórico

Não há registro confiável de quando cada animal entrou, mudou de status ou foi adotado.

PÚBLICO-ALVO E PROPOSTA

Feito para quem cuida.

Quem usa

- Voluntários de ONGs de proteção animal
- Administradores de abrigos e lares temporários
- Protetores independentes que acompanham vários resgates

O que o AdotaPet entrega

- Um cadastro único e seguro de cada animal
- Status de adoção sempre atualizado: disponível, em processo ou adotado
- Busca e filtros para encontrar rápido quem está disponível
- Acesso restrito a usuários autenticados

> “Cada adoção começa com um cadastro bem feito.”

FUNCIONALIDADES

Do login à adoção, tudo em um lugar.

Acesso seguro

Login validado no backend com JWT, cadastro de voluntário, rotas protegidas e botão Sair.

Listagem

Indicadores por status, busca por nome ou raça, filtros por espécie e status e paginação.

Cadastro e edição

Formulários validados com mensagens claras, inclusive os erros devolvidos pela API.

Detalhes e exclusão

Ficha completa do animal e exclusão sempre com diálogo de confirmação.

✅ Mensagens de sucesso e falha em todas as operações

📱 Interface responsiva e acessível

⏱️ Sessão encerrada automaticamente quando o token expira

ARQUITETURA

Três camadas, responsabilidades claras.

Frontend

ANGULAR 22 · MATERIAL

- Angular Router + lazy loading
- Serviços HTTP (Auth, Animal)
- Interceptor JWT
- Guards de rota
- Reactive Forms + signals

HTTP · JSON
Bearer JWT

Backend

SPRING BOOT 4 · JAVA 21

- Controllers REST
- Services (regras de negócio)
- Repositories (Spring Data JPA)
- DTOs + Bean Validation
- Spring Security + JWT + BCrypt

JPA
Hibernate

Banco

POSTGRESQL 17

- Tabela `usuarios`
- Tabela `animais`
- Volume Docker persistente
- Alternativa: H2 em arquivo

BACKEND · API REST

CRUD completo da entidade Animal.

```java
// AnimalController.java
@PostMapping
public ResponseEntity<AnimalResponse> criar(
        @Valid @RequestBody AnimalRequest request) {
    AnimalResponse criado = animalService.criar(request);
    URI localizacao = ServletUriComponentsBuilder
        .fromCurrentRequest().path("/{id}")
        .buildAndExpand(criado.id()).toUri();
    return ResponseEntity.created(localizacao).body(criado);
}

// AnimalRequest.java (DTO de entrada)
@NotBlank(message = "O nome é obrigatório.")
@Size(min = 2, max = 60) String nome,
```

Endpoints

`POST` /api/animais → **201** + Location
`GET` /api/animais → **200** paginado
`GET` /api/animais/{id} → **200** · **404**
`PUT` /api/animais/{id} → **200** · **404**
`DELETE` /api/animais/{id} → **204** · **404**

Erros padronizados

400 com a lista de campos inválidos, **401**, **404** e **409**, sempre em JSON pelo `GlobalExceptionHandler`.

SEGURANÇA

Autenticação com Spring Security + JWT.

1

Login

O usuário informa e-mail e senha na página de login do Angular.

2

Validação

A API compara a senha com o **hash BCrypt** salvo no banco.

3

Token

Devolve um **JWT assinado** com expiração de 2 horas. A senha nunca volta na resposta.

4

Bearer

O interceptor envia `Authorization: Bearer` em cada requisição.

5

Filtro

O `JwtAuthenticationFilter` valida assinatura e expiração antes de liberar o recurso.

`401` E-mail ou senha inválidos.

`401` Autenticação necessária.

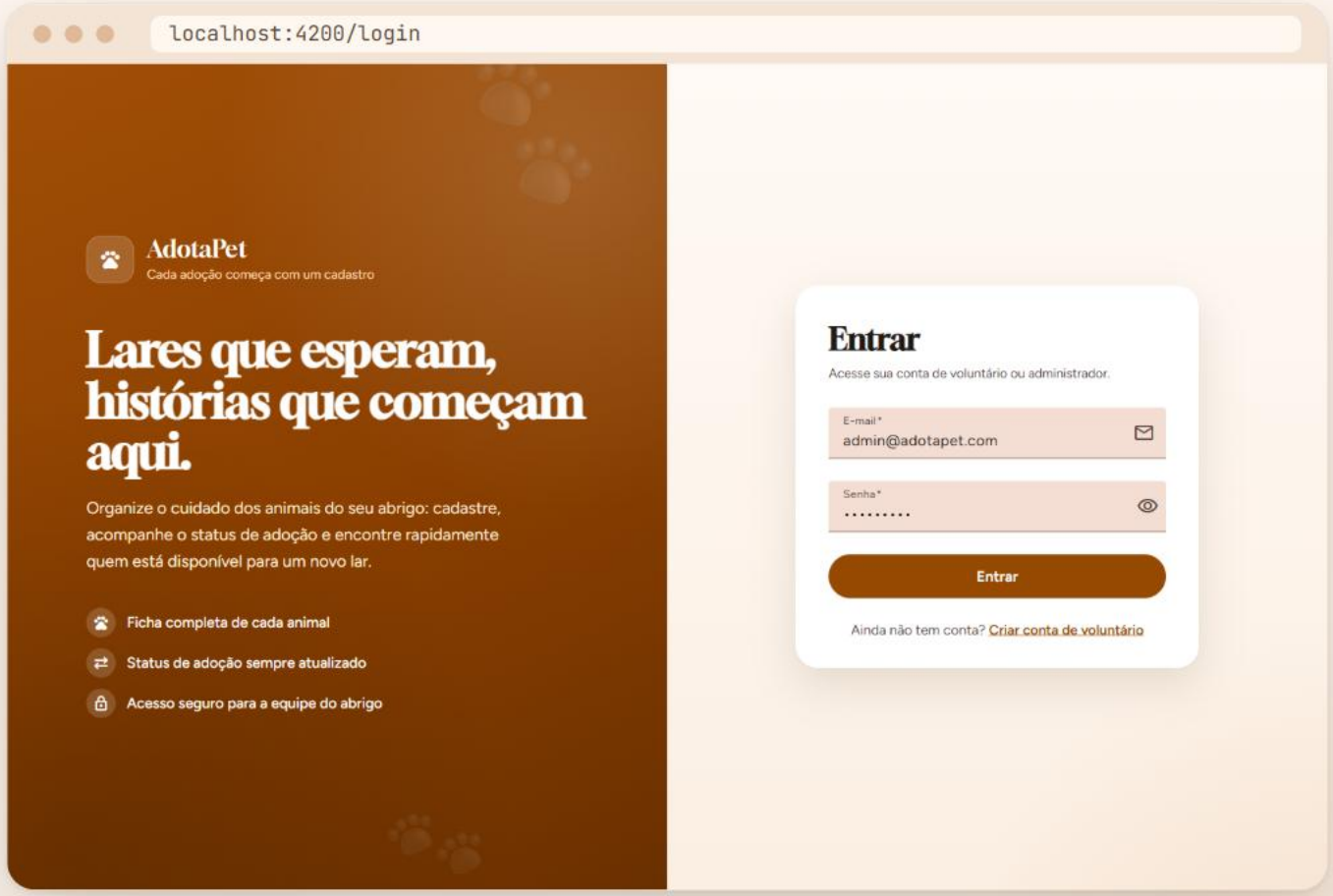
`401` Token inválido.

`401` Token expirado. Faça login novamente.

TELA · LOGIN

Entrar de verdade, no backend.

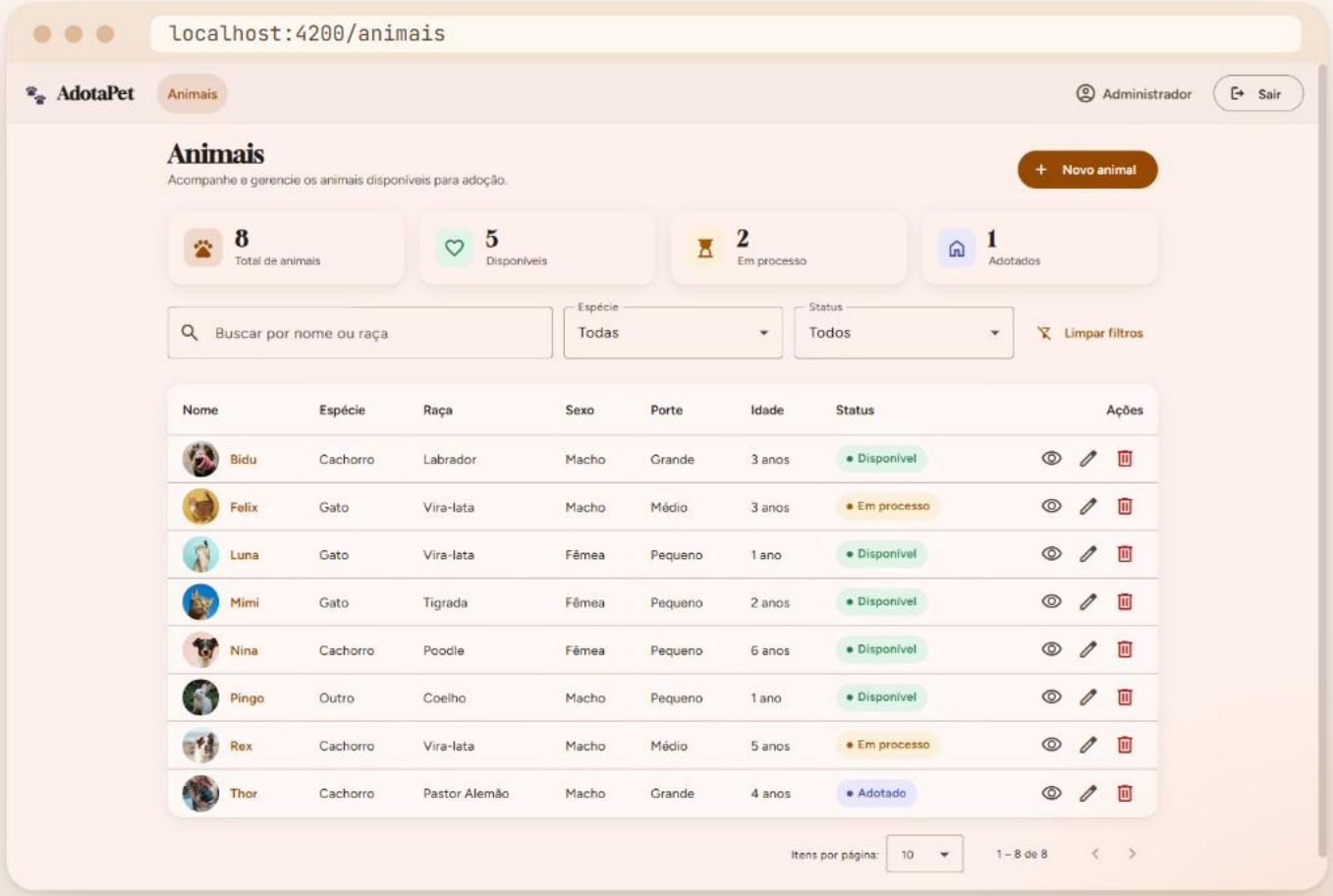
- ✓ Validação de e-mail e senha
- ✓ Erro de credenciais vindo da API
- ✓ Aviso quando a sessão expira
- ✓ Rotas protegidas redirecionam para cá



TELA · LISTAGEM

Tudo à vista.

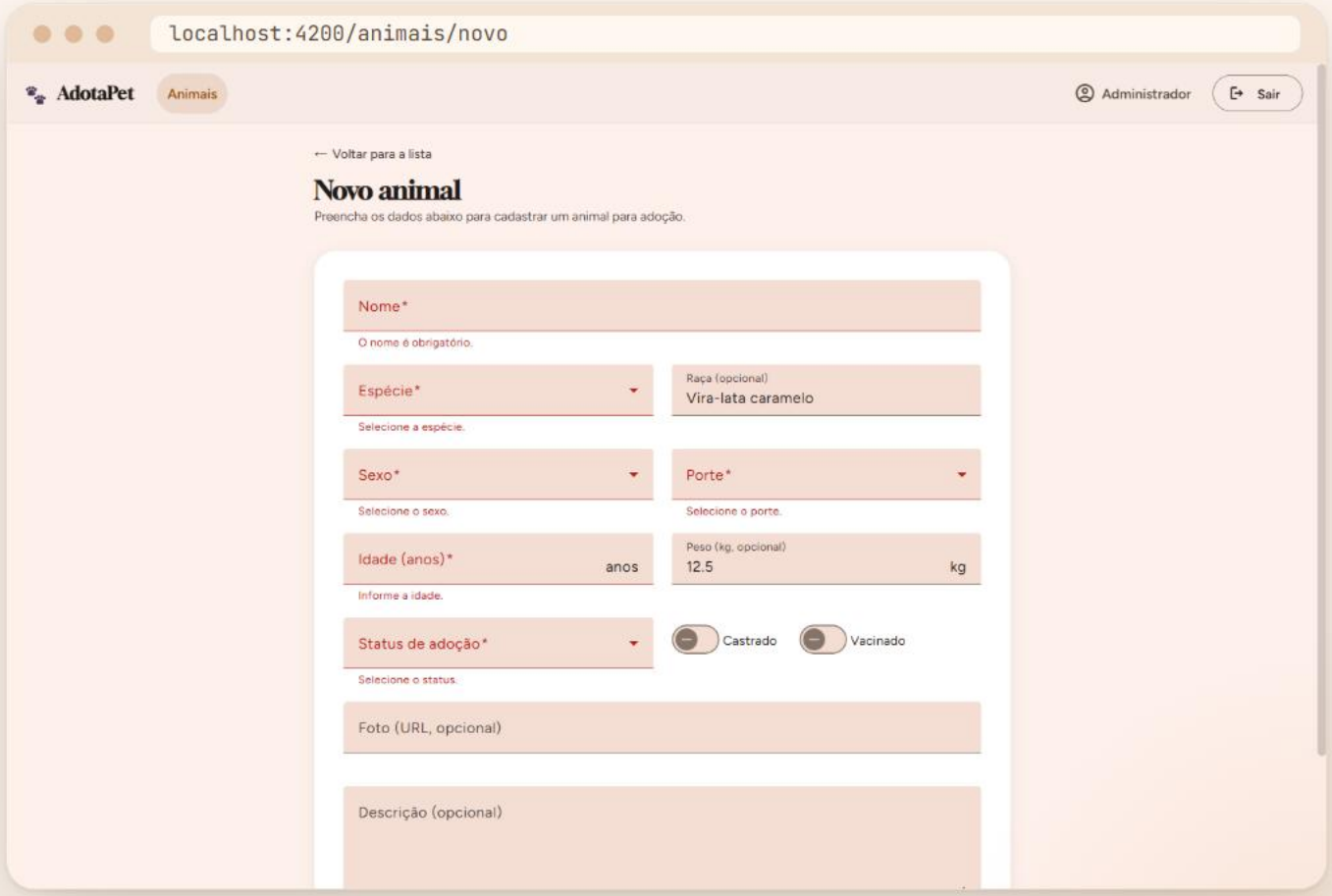
- ✓ Indicadores por status
- ✓ Busca e filtros combinados
- ✓ Paginação feita no servidor
- ✓ Ver, editar e excluir em cada linha



TELA · CADASTRO

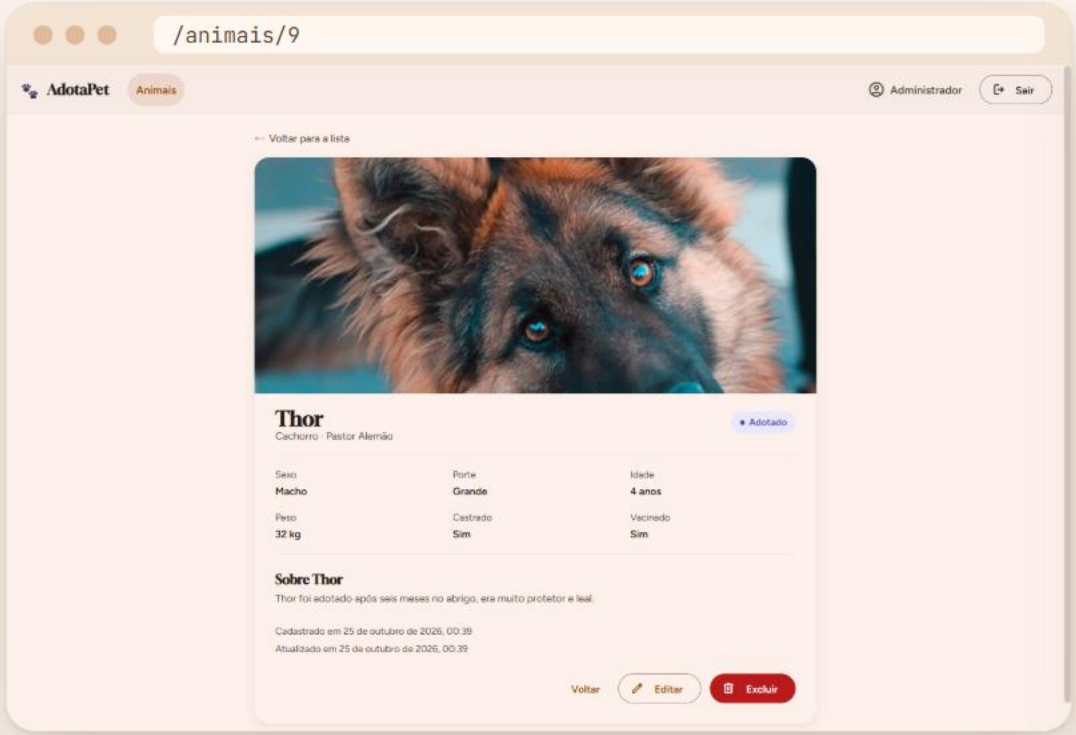
Validação que orienta.

- Mensagens claras em cada campo
- Regras iguais no Angular e na API
- Pré-visualização da foto
- Mensagem de sucesso ao salvar

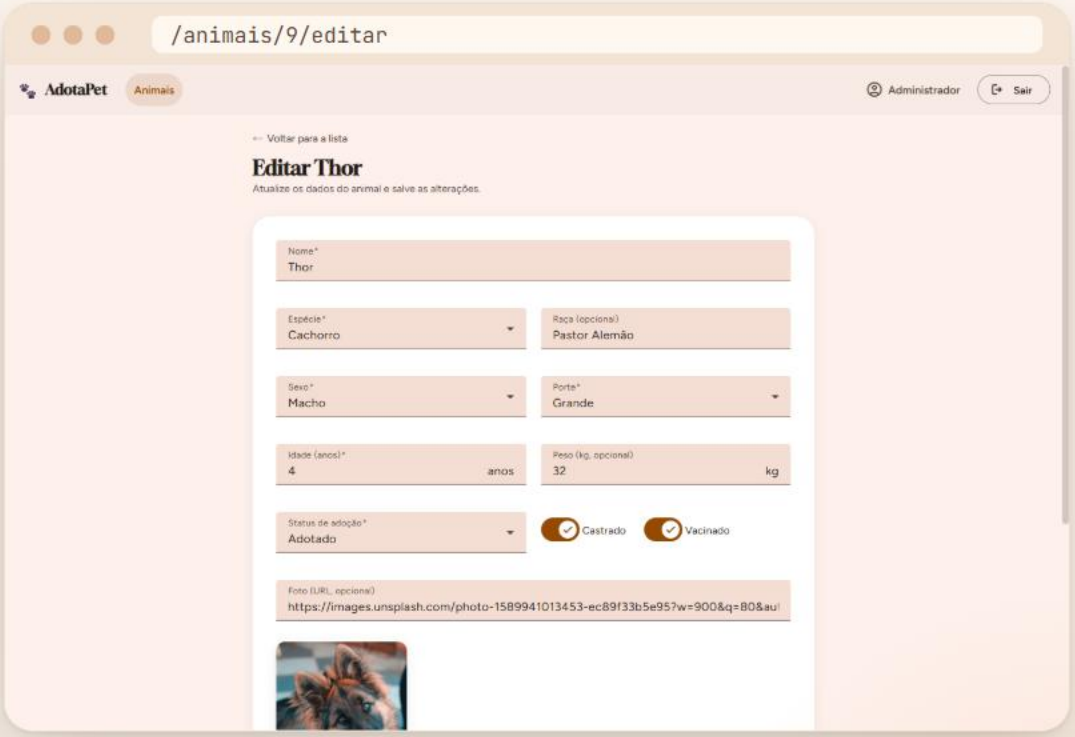


TELAS · DETALHES E EDIÇÃO

Ficha completa e edição em um clique.



Detalhes



Edição

EXCLUSÃO E RESPONSIVIDADE

Seguro e em qualquer tela.

- ✓ Diálogo de confirmação antes de excluir
- ✓ Mensagem de sucesso ou falha
- ✓ Layout adaptado para celular
- ✓ Tabela vira cards no mobile







REQUISITOS DO ENUNCIADO

Tudo o que foi pedido, entregue.

- ✓ 4 páginas + login, com Angular Router — Listagem, Cadastro, Detalhes e Edição
- ✓ Login autenticado no backend — Nada simulado no frontend
- ✓ Validação e mensagens compreensíveis — No Angular e na API, com erros 400 mapeados por campo
- ✓ Confirmação antes de excluir — Diálogo do Angular Material
- ✓ Rotas protegidas e opção de sair — authGuard e botão Sair
- ✓ Mensagens de sucesso e falha — Snackbars em todas as operações
- ✓ CRUD completo da entidade Animal — POST, GET, GET por id, PUT e DELETE
- ✓ Controllers, services e repositories — Camadas separadas, com DTOs na entrada e na saída
- ✓ Códigos HTTP adequados — 200, 201, 204, 400, 401, 404 e 409
- ✓ Spring Data JPA + banco relacional — Dados persistem após reiniciar
- ✓ JWT com expiração + Bearer — Token inválido ou expirado tratado nos dois lados
- ✓ Senhas com BCrypt — Nunca devolvidas pela API

QUALIDADE

Testado de ponta a ponta.

28

testes no backend (unitários e de integração com MockMvc)

9

endpoints REST documentados no Swagger

6

páginas no Angular, todas responsivas

0

senhas expostas: hash BCrypt e DTOs sem senha

Testes automatizados

Login, token inválido e expirado, CRUD completo, 400, 404 e 409, além dos serviços e guards do Angular.

Persistência validada

Backend reiniciado e dados mantidos no PostgreSQL, também no perfil H2 em arquivo.

Fácil de rodar

`docker compose up -d`, `mvnw spring-boot:run` e `npm start`.

EQUIPE

Quem construiu o AdotaPet.

TB

Társis Carvalho Barreto

202220038

Infraestrutura, integração e autenticação no frontend

@Tarsis59

YB

Ylo Silva de Sá Bittencourt

202220039

Segurança: Spring Security, JWT e BCrypt

@YloBittencourt

GF

Guilherme Crocamo Freire

202220448

Entidades, CRUD, DTOs, validação e erros

@gcrocamo

SS

Samuel Ferreira Santos

202410518

Listagem, detalhes e exclusão no Angular

@SamuelFerrc

JO

João Pedro França Chaves Oliveira

202320100

Cadastro, edição e formulários no Angular

@joao-pedro-franca

DEMONSTRAÇÃO

Obrigado!

Agora, a demonstração ao vivo: login, navegação pelas páginas e todas as operações do CRUD.

Repositório	`github.com/Tarsis59/adotapet`
Aplicação	`localhost:4200`
API · Swagger	`localhost:8080/swagger-ui.html`
Login demo	`admin@adotapet.com · Admin@123`